



ambient
scientific

Ambient Scientific Inc.

AHB_I2C_INTERFACE

CONFIDENTIAL

Version : 0.6 Date : 12/11/2021 Author : CH Pavan Kumar Singh Abhishek B Revankar Gabbar Jadav

Confidentiality Statement

This document contains highly confidential and proprietary information and data of Ambient Scientific and its affiliates. This document is provided, on the express condition that such information and data will not be used, disclosed, or reproduced in any form, in whole or in part, for any purpose (other than solely for evaluation purposes by authorized representatives with a need to know), without the express written approval of Ambient Scientific. The right to use, disclose, and reproduce such information and data shall be governed by the service agreement between the client and Ambient Scientific

Warnings and Disclaimer

The information provided is on an as is basis. Every effort has been made to make this book as complete and as accurate as possible, but no warranty or fitness is implied. The authors and the publisher shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained in this book. Screenshots and data used within are for illustration purpose and may vary from the actual application

Revision History	Date	Author	Details
0.1	23-11-2020	Abhishek Pavan	Release of Document
0.2	16/12/2020	Abhishek Pavan	Updated Soft_reset (SW) signal & Default values of Registers.
0.3	07/01/2021	Abhishek Pavan	Added I2C_BEGIN_TRANSFER register for enabling I2C_MASTER and Software flow for programming registers for I2C Master and I2C Slave.
0.4	08/01/2021	Abhishek Pavan	Added Masked Interrupts for I2C
0.5	18/01/2021	Abhishek Pavan	Added Re-start feature for I2C master and status register regarding Acknowledge.
0.6	01/02/2021	Abhishek Pavan	Added De-bug Bus feature.

Note: This document has details and information regarding I2C SLAVE also. Since I2C SLAVE is not present in S10, Kindly ignore any information regarding I2C SLAVE (i.e., Register description, Configuration and programming modes etc.)

INDEX

Content	Page No
Introduction	05
Signal List	06
Register Banks	10
Interrupts	22
Configuring Registers	23
Block Diagram	25
AHB I2C Flow	29
I2C Protocol	30
Software Rest	31
Reference	32

1. Introduction

The I2C bus is a two-wire serial interface, consisting of a serial data line (SDA) and a serial clock (SCL). These wires carry information between the devices connected to the bus. Each device is recognized by a unique address and can operate as either a “transmitter” or “receiver,” depending on the function of the device. Devices can also be considered as masters or slaves when performing data transfers. A master is a device that initiates a data transfer on the bus and generates the clock signals to permit that transfer. At that time, any device addressed is considered a slave. Following modes can be operated on a I2C bus

- Standard Mode operates up to 100kb/s
- Fast mode operates up to 400kb/s
- Fast mode plus operates up to 1Mb/s
- High speed mode operates up to 3.4Mb/s
- Ultra Fast mode operates up to 5Mb/s and is uni-directional.

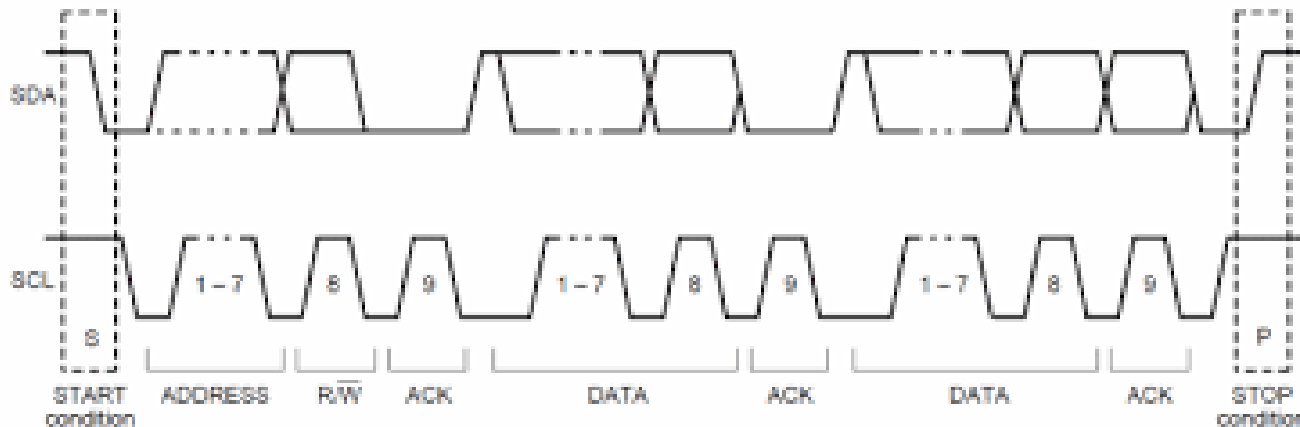


Fig 1: Basic communication on I2C Bus

2. Signal List

Name	Source	Description
HWDATA[31:0] Write data bus	MASTER	The write data bus is used to transfer data from the master to the bus slaves during write operations. A minimum data bus width of 32 bits is recommended. However, this may easily be extended to allow for higher bandwidth operation. Note: Here we are making use of 32 bits
HSEL Slave select	MASTER	Each AHB slave has its own slave select signal and this signal indicates that the current transfer is intended for the selected slave. This signal is simply a combinatorial decode of the address bus.
HRDATA[31:0] Read data bus	SLAVE	The read data bus is used to transfer data from bus slaves to the bus master during read operations. A minimum data bus width of 32 bits is recommended. However, this may easily be extended to allow for higher bandwidth operation. Note: Here we are making use of 32 bits

HREADY Transfer done	MASTER	When HIGH the HREADY signal indicates that a transfer has finished on the bus or ready to transfer the next data.
HRESP[1:0] Transfer response	Slave	The transfer response provides additional information on the status of a transfer. Four different responses are OKAY, ERROR, RETRY and SPLIT. Note: Here we are making use of only the OKAY response.
HCLK System clock	MASTER	The rising edge of HCLK is used to time all transfers on the AHB
HRESETn AHB reset	MASTER	The AHB bus reset signal is active LOW and this signal will normally be connected directly to the system bus reset signal.
HTRANS[1:0] Transfer type	MASTER	Every transfer can be classified into one of four different types, as indicated by the HTRANS[1:0] signals – IDLE, BUSY, NONSEQ & SEQ
HSIZE[2:0] Transfer size	MASTER	Indicates the size of the transfer.
HWRITE AHB transfer direction	MASTER	When HIGH this signal indicates an AHB write access and when LOW a read access.
HADDR[31:0] AHB address bus	MASTER	This is the AHB address bus, which may be up to 32-bits wide.
HREADYOUT Device ready signal	SLAVE	Signal sent to master to indicate that the device is ready to accept the data.

i2c_sda_in	I2C Master	Input signal to I2C master from PADS
i2c_sda_out	I2C Master	Output signal from I2C MASTER which is sent out through PADS
i2c_scl	I2C Master	Output clock signal from I2C MASTER which is sent out through PADS
i2c_sda_slave_in	I2C Slave	Input signal to I2C SLAVE from PADS
i2c_sda_slave_out	I2C Slave	Output signal from I2C SLAVE which is sent out through PADS
i2c_scl_slave_in	I2C Slave	Input clock signal to I2C SLAVE from PADS
I2CM_sw_rstn (Have same functionality as HRESETn)	I2C Master	Active low software reset signal for I2C MASTER.
I2CS_sw_rstn (Have same functionality as HRESETn)	I2C Slave	Active low software reset signal for I2C SLAVE.
lfrs (length for repeated start)	I2C Master	Input from AHB to I2C TOP which indicates length of transfer if re-start is enabled
re_start_enb	I2C Master	Input coming from AHB to I2C TOP which indicates re-start is enabled or not
repeat_we_re	I2C Master	Indicated whether AHB wants to do read/write transfer if re-start is enabled
master_ack_detect	I2C Master	Indicates the acknowledge status of I2C MASTER
master_no_ack_detect	I2C Master	Indicates the no_acknowledge status of I2C MASTER

debug_i2c_master	I2C Master	Output signal of I2C master debug bus
debug_i2c_slave	I2C Slave	Output signal of I2C slave debug bus
debug_i2c_master_tx	I2C Master TX_FIFO	Output signal of I2C master TX FIFO debug bus
debug_i2c_master_rx	I2C Master RX_FIFO	Output signal of I2C master RX FIFO debug bus
debug_i2c_slave_tx	I2C Slave TX_FIFO	Output signal of I2C slave TX FIFO debug bus
debug_i2c_slave_rx	I2C Slave RX_FIFO	Output signal of I2C slave RX FIFO debug bus
debug_ahb_i2c	AHB Slave	Output signal of AHB slave
Slave_addr_top	I2C Master	Address of the slave connected to I2C bus (must be taken from external slave document connected on bus)
Length_top	I2C Master	Number of bytes to be transmitted/received on i2c bus
func_debug_mode_en	AHB I2C	Helps in functional debug bus enable
masked_i2c_slave_interrupt	AHB I2C	Interrupt value of I2C Slave
masked_i2c_master_interrupt	AHB I2C	Interrupt value of I2C Master
Debug bus 0 - Debug bus 7	AHB I2C	Debug bus values of all internal instance modules

Table 1: Signal List present on AHB and I2C bus

3. Register Banks

I2C Master Address Range : 32'h5000_2400 to 32'h5000_27FF			
S.No	Register Name	Register Address	Default Value
1	I2C CTRL REGISTER	32'h 5000_2400	32'h 0000_0000
2	I2C TX FIFO STATUS	32'h 5000_2404	32'h 0000_001D
3	I2C RX FIFO STATUS	32'h 5000_2408	32'h 0000_001D
4	I2C TX DATA	32'h 5000_240C	32'h 0000_0000
5	I2C RX DATA	32'h 5000_2410	32'h 0000_0000
6	I2C BEGIN TRANSFER	32'h 5000_2418	32'h 0000_0000
7	I2C MASTER MASK REGISTER	32'h 5000_241C	32'h 0000_7F7F
8	I2C STATUS REGISTER	32'h 5000_2420	32'h 0000_000F
9	I2C DIVIDER	32'h 5000_2414	32'h 0000_0002

Table 2: Register Bank of I2C Master

I2C Slave Address Range : 32'h5000_2800 to 32'h5000_2BFF			
S.No	Register Name	Register Address	Default Value
1	I2C SLAVE TX FIFO STATUS	32'h 5000_2800	32'h 0000_001D
2	I2C SLAVE RX FIFO STAUS	32'h 5000_2804	32'h 0000_0001
3	I2C SLAVE TX DATA	32'h 5000_2808	32'h 0000_0000
4	I2C SLAVE RX DATA	32'h 5000_280C	32'h 0000_0000
5	I2C SLAVE MASK REGISTER	32'h 5000_2810	32'h 0000_0F7F

Table 3: Register Bank of I2C Slave

Note: Apart from the Address mentioned in the above table, all the other address range is not used till now. Unused address location reading should be getting zero

I2C CONTROL REGISTER			
Bit	Name	R/W	Description
[5:0]	Length	W/R	Indicates number of bytes to be transmitted / received on i2c bus
[15:8]	Slave Address	W/R	Address of the slave present on i2c bus along with read/write bit (must be taken from slave document connected on to i2c bus)
[16]	i2c_master_far_near_enb	W/R	Indicates weather far end or near end debug mode is selected Description of debug fifo mode along with far end and near end is given below the table (For both master and slave)
[17]	i2c_master_debug_enb	W/R	Indicates weather debug mode is enabled or not for I2C master
[18]	i2c_slave_far_near_enb	W/R	Indicates weather far end or near end debug mode is selected
[19]	i2c_slave_debug_enb	W/R	Indicates weather debug mode is enabled or not for I2C master
[20]	Repeat start	W/R	Bit for enabling repeat start for i2c master '0' – No repeat start, '1' – Repeat start. Note : Refer I2C protocol for repeat start feature, Link to document is given in reference section
[26:21]	Length	W/R	Indicates number of bytes to be transmitted / received on i2c bus after repeat start
[27]	Read/Write	W/R	Bit for Read/Write Operation in repeat start '0' – Write Operation, '1' – Read Operation.
[28]	Reset for external slave	W/R	Indicates weather external slave reset command is given or not '0' : Reset command is made low. '1' : Reset command is made high. Description is given below this register table
[7:6] [31:29]	Reserved	-	Always reads "0"

Table 4: I2C CONTROL REGISTER DESCRIPTION

Debug FIFO Logic

This logic is mainly used to transfer the data from TX FIFO to RX FIFO and vice versa based on debug enable and far_near_enb bits described in the above table (I2C CONTROL REGISTER description).

Far End Mode : In this mode we can check if data available TX FIFO memory is being stored in RX FIFO memory or not

Near End Mode : In this mode we can check if data available RX FIFO memory is being stored in TX FIFO memory or not

It mainly helps us in debugging fifo memory and check if data is properly storing in both FIFO's memory properly or not.

Signals i2c_master_far_near_enb, i2c_master_debug_enb, i2c_slave_far_near_enb, i2c_slave_debug_enb (bits 16, 17, 18, 19) described in I2C CONTROL REGISTER description helps in controlling debug fifo logic

By default debug enable signal and far_near_enb signal will be low for both master and slave.

For enabling i2c master debug fifo logic bit i2c_master_debug_enb (bit 17) must be made high and depending upon far end or near end debug to be performed bit i2c_master_far_near_enb (bit 16) must be set

- If bit i2c_master_far_near_enb is set as “0” after making i2c_master_debug_enb high, Near end mode can be checked
- If bit i2c_master_far_near_enb is set as “1” after making i2c_master_debug_enb high, Far end mode can be checked

For enabling i2c slave debug fifo logic bit i2c_slave_debug_enb (bit 19) must be made high and depending upon far end or near end debug to be performed bit i2c_slave_far_near_enb (bit 18) must be set

- If bit i2c_slave_far_near_enb is set as “0” after making i2c_slave_debug_enb high, Near end mode can be checked
- If bit i2c_slave_far_near_enb is set as “1” after making i2c_slave_debug_enb high, Far end mode can be checked

The **Reset external slave** bit (bit [28]) is applicable only if external slave connected to bus is EEPROM only, when asserted a sequence is followed as described in EEPROM document, Link for the document is given in reference section.

I2C TX DATA			
Bit	Name	R/W	Description
[31:0]	TX DATA	W/R	Data which has to be transferred on to I2C bus will be stored in TX FIFO memory through I2C TX DATA Register from AHB Care should be taken to avoid over_run (Refer TX FIFO STATUS Register) while storing into fifo, Reading this register will result in getting garbage value

Table 5: I2C TX DATA Register Description

I2C RX DATA			
Bit	Name	R/W	Description
[31:0]	RX DATA	R	Data read from I2C bus will be stored in RX FIFO memory, from there AHB can read that data through this register Care should be taken to avoid under_run (Refer RX FIFO STATUS Register) while reading from fifo, writing this register will result in effecting other register

Table 6: I2C RX DATA Register Description

I2C SLAVE TX DATA			
Bit	Name	R/W	Description
[31:0]	SLAVE TX DATA	W/R	Data which has to be transferred on to I2C bus will be stored in TX FIFO memory through this Register from AHB Care should be taken to avoid over_run (Refer SLAVE TX FIFO STATUS Register) while storing into fifo, Reading this register will result in getting garbage value

Table 7: I2C SLAVE TX DATA Register Description

I2C SLAVE RX DATA			
Bit	Name	R/W	Description
[31:0]	SLAVE RX DATA	R	Data read from I2C bus will be stored into SLAVE RX FIFO memory, from there AHB can read that data through this register Care should be taken to avoid under_run (Refer SLAVE RX FIFO STATUS Register) while reading from fifo, writing this register will result in effecting other register

Table 8: I2C SLAVE RX DATA Register Description

I2C BEGIN TRANSFER			
Bit	Name	R/W	Description
[0]	Begin Transfer	W/R	This bit will enable I2C master to begin transfer data on to the I2C Bus

Table 9: I2C BEGIN TRANSFER Register Description**Note:**

1. Before enabling this register make sure all the steps mentioned in configuring I2C master has been taken care, this register should only be used when user wants to use I2C Master.
2. Once enabled manual disabling is not possible, I2C master will disable once the transfer has been done as per command given through I2C CTRL Register

I2C DIVIDER REGISTER			
Bit	Name	R/W	Description
[31:0]	Scaling Value	W/R	Used for generating SCL as per scaling value set by the user.

Table 10: I2C DIVIDER Register Description**Note:**

1. Scaling value can't be set to 0, if given value is "0" default scaling value (which is divide by 2 currently) will be taken for SCL generation
2. Entered scaling value must be EVEN, if ODD value is entered the generated clock will be equal to its preceding even value, if odd number is entered it will be rounded off to its nearest even number Ex: if 5 is entered it will be rounded off to 4

I2C TX FIFO STATUS			
Bit	Name	R/W	Description
[0]	Empty	R	Indicates “EMPTY” status of TX FIFO
[1]	Full	R	Indicates “FULL” status of TX FIFO
[2]	At least 4 bytes	R	Indicates at least 4 bytes of data can be written into the TX FIFO memory
[3]	At least 2 bytes	R	Indicates at least 2 bytes of data can be written into the TX FIFO memory
[4]	At least 1 byte	R	Indicates at least 1 bytes of data can be written into the TX FIFO memory
[5]	Overrun	R/W1C	Indicates that the FIFO is being over written even after full condition. Write ‘1’ to clear the bit if over run is asserted Note: Care should be taken to avoid overrun.
[6]	Under run	R/W1C	Indicates that the FIFO is read when it is in empty condition. Write ‘1’ to clear the bit if under run is asserted Note: Care should be taken to avoid under run.
[11:8]	Read Pointer	R	Indicates read pointer value
[19:16]	Write Pointer	R	Indicates write pointer value
[27:24]	Status count	R	Indicates the status count of FIFO for byte availability.
[7], [12:15], [20:23], [28:31]	Reserved	-	Always reads “0”

Table 11: I2C TX FIFO STATUS Register Description**Note :**

1. Writing data into TX FIFO memory has to be done by constant monitoring of this register (based on full, atleast 4 bytes, atleast 2 bytes, atleast 1 bytes).

I2C RX FIFO STATUS			
Bit	Name	R/W	Description
[0]	Empty	R	Indicates “EMPTY” status of RX FIFO
[1]	Full	R	Indicates “FULL” status of RX FIFO
[2]	At least 4 bytes	R	Indicates at least 4 bytes of data can be read from RX FIFO memory
[3]	At least 2 bytes	R	Indicates at least 2 bytes of data can be read from RX FIFO memory
[4]	At least 1 byte	R	Indicates at least 1 bytes of data can be read from RX FIFO memory
[5]	Overflow	R/W1C	Indicates that the FIFO is being over written even after full condition. Write ‘1’ to clear the bit if over run is asserted Note: Care should be taken to avoid overflow.
[6]	Under run	R/W1C	Indicates that the FIFO is read when it is in empty condition. Write ‘1’ to clear the bit if under run is asserted Note: Care should be taken to avoid under run.
[11:8]	Read Pointer	R	Indicates read pointer value
[19:16]	Write Pointer	R	Indicates write pointer value
[27:24]	Status count	R	Indicates the status count of FIFO for byte availability.
[7], [12:15], [20:23], [28:31]	Reserved	-	Always reads “0”

Table 12: I2C RX FIFO STATUS Register Description

Note :

1. Reading data from RX FIFO memory has to be done by constant monitoring of this register

I2C SLAVE TX FIFO STATUS			
Bit	Name	R/W	Description
[0]	Empty	R	Indicates “EMPTY” status of SLAVE TX FIFO
[1]	Full	R	Indicates “FULL” status of SLAVE TX FIFO
[2]	At least 4 bytes	R	Indicates at least 4 bytes of data can be written into the SLAVE TX FIFO memory
[3]	At least 2 bytes	R	Indicates at least 2 bytes of data can be written into the SLAVE TX FIFO memory
[4]	At least 1 byte	R	Indicates at least 1 bytes of data can be written into the SLAVE TX FIFO memory
[5]	Overrun	R/W1C	Indicates that the FIFO is being over written even after full condition. Write ‘1’ to clear the bit if over run is asserted Note: Care should be taken to avoid overrun.
[6]	Under run	R/W1C	Indicates that the FIFO is read when it is in empty condition. Write ‘1’ to clear the bit if under run is asserted Note: Care should be taken to avoid under run.
[11:8]	Write Pointer	R	Indicates read pointer value
[15:12]	Read Pointer	R	Indicates write pointer value
[7], [31:16]	Reserved	-	Always reads “0”

Table 13: I2C SLAVE TX FIFO STATUS Register Description

Note :

1. Writing data into SLAVE TX FIFO memory has to be done by constant monitoring of this register

I2C SLAVE RX FIFO STATUS			
Bit	Name	R/W	Description
[0]	Empty	R	Indicates “EMPTY” status of SLAVE RX FIFO
[1]	Full	R	Indicates “FULL” status of SLAVE RX FIFO
[5]	Overrun	R/W1C	Indicates that the FIFO is being over written even after full condition. Write ‘1’ to clear the bit if over run is asserted Note: Care should be taken to avoid overrun.
[6]	Under run	R/W1C	Indicates that the FIFO is read when it is in empty condition. Write ‘1’ to clear the bit if under run is asserted Note: Care should be taken to avoid under run.
[11:8]	Read Pointer	R	Indicates read pointer value
[15:12]	Write Pointer	R	Indicates write pointer value
[2:4], [7], [31:16]	Reserved	-	Always reads “0”

Table 14: I2C SLAVE RX FIFO STATUS Register Description

Note :

1. Reading data from SLAVE RX FIFO memory has to be done by constant monitoring of this register

I2C MASTER MASK REGISTER			
Bit	Name	R/W	Description
[6:0]	TX_MASKED_reg	R/W	This is used for Masking the Interrupts, If it is '1' Interrupt is Masked, If it is '0' Interrupt is not Masked. 0 th bit is used for empty, 1 st bit is used for full, 2 nd bit is used for at least 4 bytes, 3 rd bit is used for at least 2 bytes, 4 th bit is used for at least 1 byte, 5 th bit used for Over run, 6 th bit is used for Under run.
[7]	Reserved	-	Always reads "0"
[14:8]	RX_MASKED_reg	R/W	This is used for Masking the Interrupts, If it is '1' Interrupt is Masked, If it is '0' Interrupt is not Masked. 8 th bit is used for empty, 9 th bit is used for full, 10 th bit is used for at least 4 bytes, 11 th bit is used for at least 2 bytes, 12 th bit is used for at least 1 byte, 13 th bit used for Over run, 14 th bit is used for Under run.
[31:15]	Reserved	-	Always reads "0"

Table 15: I2C MASTER MASK Register Description

I2C SLAVE MASK REGISTER			
Bit	Name	R/W	Description
[6:0]	TX_MASKED_reg	R/W	This is used for Masking the Interrupts, If it is '1' Interrupt is Masked, If it is '0' Interrupt is not Masked. 0th bit is used for empty, 1st bit is used for full, 2nd bit is used for at least 4 bytes, 3rd bit is used for at least 2 bytes, 4th bit is used for at least 1 byte, 5th bit used for Over run, 6th bit is used for Under run.
[7]	Reserved	-	Always reads "0"
[11:8]	RX_MASKED_reg	R/W	This is used for Masking the Interrupts, If it is '1' Interrupt is Masked, If it is '0' Interrupt is not Masked. 8th bit is used for empty, 9th bit is used for full, 10th bit used for Over run, 11th bit is used for Under run.
[31:12]	Reserved	-	Always reads "0"

Table 16: I2C MASTER MASK Register Description

I2C STATUS REGISTER			
Bit	Name	R/W	Description
[1:0]	No ACK Status	R/W11C	Indicates the No acknowledge status from I2C Bus.
[3:2]	ACK Status	R/W11C	Indicates the Acknowledge status from I2C Bus.
[9:4]	Bytes Transferred	R	Indicates Number of byte transferred onto I2C bus based on Length (For Length Refer I2 control REG description)
10	Clear Bytes	R/W	For making bytes transferred bits to zero
[31:11]	Reserved	R	Always reads “0”

Table 17: I2C MASTER MASK Register Description

Value	Acknowledge	No-Acknowledge
00	Acknowledgment received during Address phase	No-Acknowledgment received during address phase
01	Acknowledgment received during Write operation	No-Acknowledgment received during Write operation
10	Acknowledgment sent during read operation	No-Acknowledgment received during read operation
11	Default Value	Default Value

Table 18: Description of values in I2C STATUS REG FOR NO_ACK AND ACK BITS

Note : The above register gives the acknowledge status of I2C bus while read/write operations. Once ACK/NO_ACK is detected a clear signal has to be given for setting them to default Value

Bytes Transferred: Consider a length of 5 has been initiated on I2C bus, and we received No_Ack after a particular byte. This feature helps us in debugging the exact number of transfers completed before No_Ack received on I2C Bus. A clear signal has to be given before start of new transfer.

4. Interrupts

An interrupt is a signal to the processor emitted by hardware or software indicating an event that needs immediate attention. Whenever an interrupt occurs, the controller completes the execution of the current instruction and starts the execution of an Interrupt Service Routine (ISR) or Interrupt Handler. ISR tells the processor or controller what to do when the interrupt occurs. The interrupts can be either hardware interrupts or software interrupts.

Masking of Interrupts

An Interrupt that can be disabled or ignored by the instructions of CPU is called as Maskable Interrupt. There are several interrupts coming from the IP block which are masked based on the configurable register provided by the AHB Slave.

The below figure describes the masking of interrupts, the masked value is loaded from AHB_Slave through I2C_MASTER_MASKED_REG for i2c master and I2C_SLAVE_MASKED_REG for i2c slave. To mask the respective interrupt signals load it with value of 1 and to unmask it load it with 0.

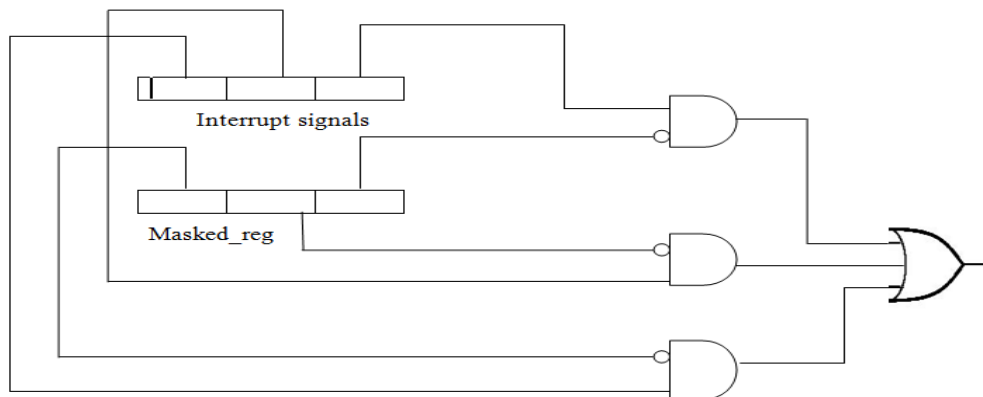


Fig 2: Masking of Interrupt signals

Block	Interrupt Number
I2C Master	44
I2C Slave	43 [Removed for S10]

Table 19: Interrupt signal number for master and slave of I2C

Note: Refer Masked Registers for both master and slave described in Register banks chapter

5. Configuring Registers

5.1 Master Mode

I2C Master Supports following modes of operation on I2C Bus

- **Standard Mode (up to 100KHz)**
- **Fast mode (100KHz to 400KHz)**
- **Fast Mode Plus(400KHz to 1MHz)**

When operating I2C Master for transferring data onto I2C bus following steps has to be taken care

- Load the registers with required data based on the register description given in the register banks chapter
- While loading slave address in I2C CTRL REG refer the external slave document for slave address, along with slave address load the read/write bit also in the register, this will indicate master either to transmit/receive the data onto/from the I2C bus
- If data transmission (write operation) onto the i2c bus has to be performed load the required data to be sent, into the TX FIFO memory through I2C TX DATA Register following I2C TX FIFO STATUS register description given above in register banks. Avoid Over run while loading data and LSB bit in the slave address section of I2C CTRL REG must be “0” as per protocol.
- If data receiving (read operation) from the i2c bus has to be performed set the LSB bit of the slave address section of I2C CTRL REG must be “1” as per protocol. When trying to read received data from RX FIFO memory check I2C RX FIFO STATUS register for availability of data based on empty or atleast 4 bytes, atleast 2 bytes and atleast 1 bytes and read using I2C RX DATA Register.
- Load the I2C DIVIDER register with the required SCL clock frequency based on the mode of operation (i.e., Standard mode/Fast mode/Fast mode plus)
- Enable begin transfer register only after all the above steps has been done and required registers have been loaded properly.

Note:

- Once begin transfer is enabled I2C CTRL REG cannot be changed as it would result in improper function of I2C Master, Software protection is given such that once after enabling begin transfer I2C CTRL REG cannot be changed, New control value can only be loaded once ongoing transfer is completed.
- I2C Master only supports 7-bit addressing slave only.
- During read operation, Master will generate NO_ACK while reading last byte from slave.

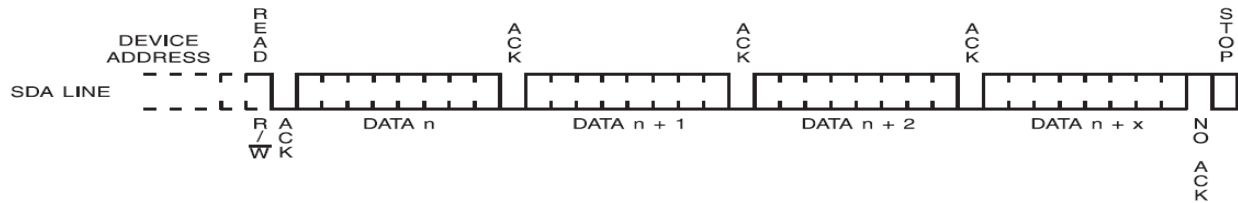


Fig 3: Transfer structure of I2C Protocol

5.2 Slave Mode

When we are being operated as I2C Slave in the I2C bus following steps has to be taken care

The 7-bit slave address is 0x0011010 and slave address will be 0x0011010x

When LSB bit (x) is 0 it indicates external master wants to perform write operation with our slave, if it is 1 it indicated external master wants to perform read operation with our slave

- When external master is transmitting data into the slave, the received data will be stored in SLAVE RX FIFO memory. The stored data can be read through AHB by following I2C SLAVE RX FIFO STATUS reg.
- When external master wants to do a read operation, the data loaded in the TX FIFO memory will be sent out onto the SDA line of I2C bus, data must be stored into SLAVE TX FIFO memory following SLAVE TX FIFO STATUS reg.
- If the address sent by external master does not match with our slave address, NO_ACK will be given onto I2C bus.
- Slave will recognize the NO_ACK sent from master as a end of read operation, and will stop sending data onto bus

Note:

1. Both TX and RX FIFO's used for I2C Slave are asynchronous where as FIFO's used for master are synchronous in nature.
2. A two flop synchronizer has been added for proper synchronization between i2c slave and external master signal as both works on different clock sources.

6. Block Diagram

AHB I2C TOP

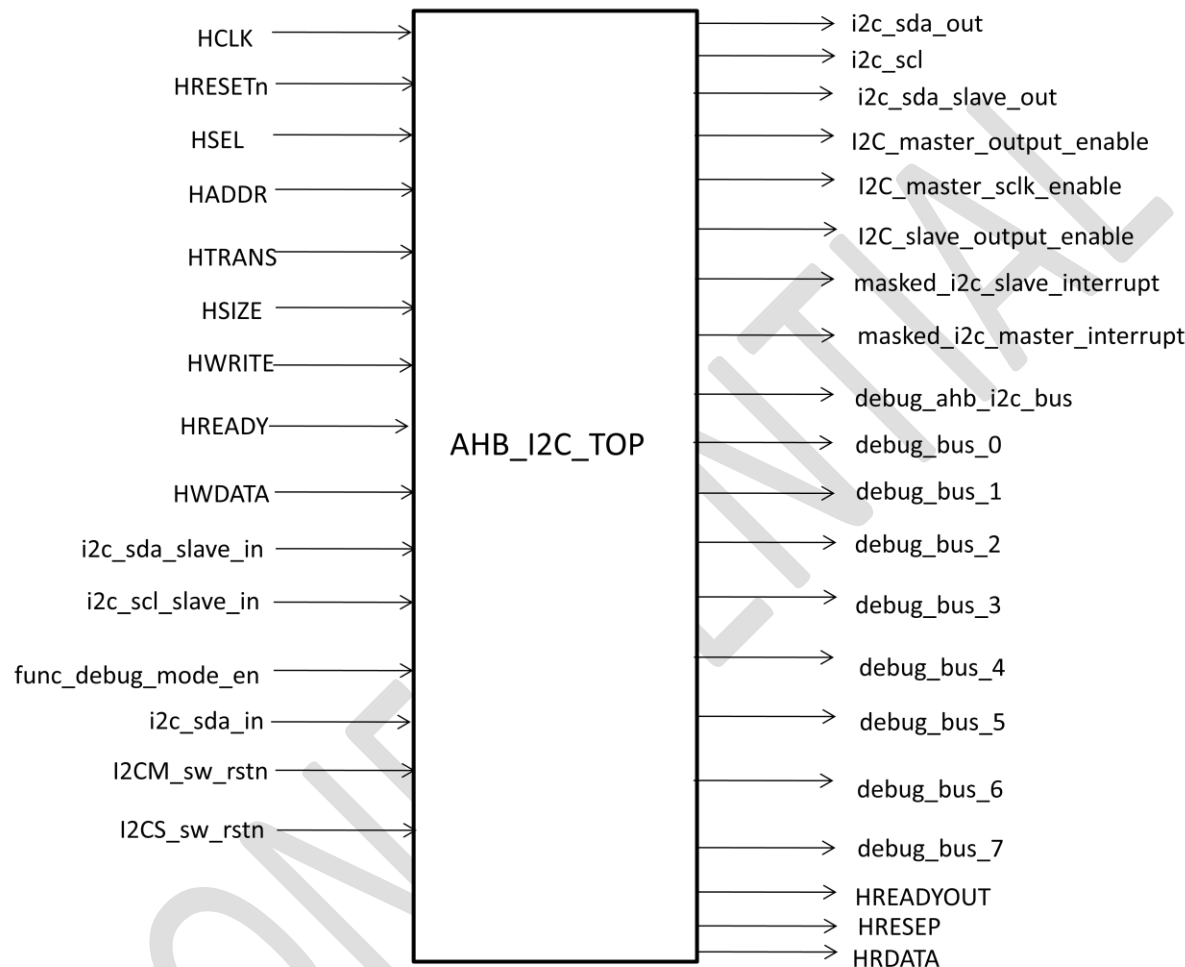


Fig 4: AHB I2C TOP

AHB SLAVE INTERFACE

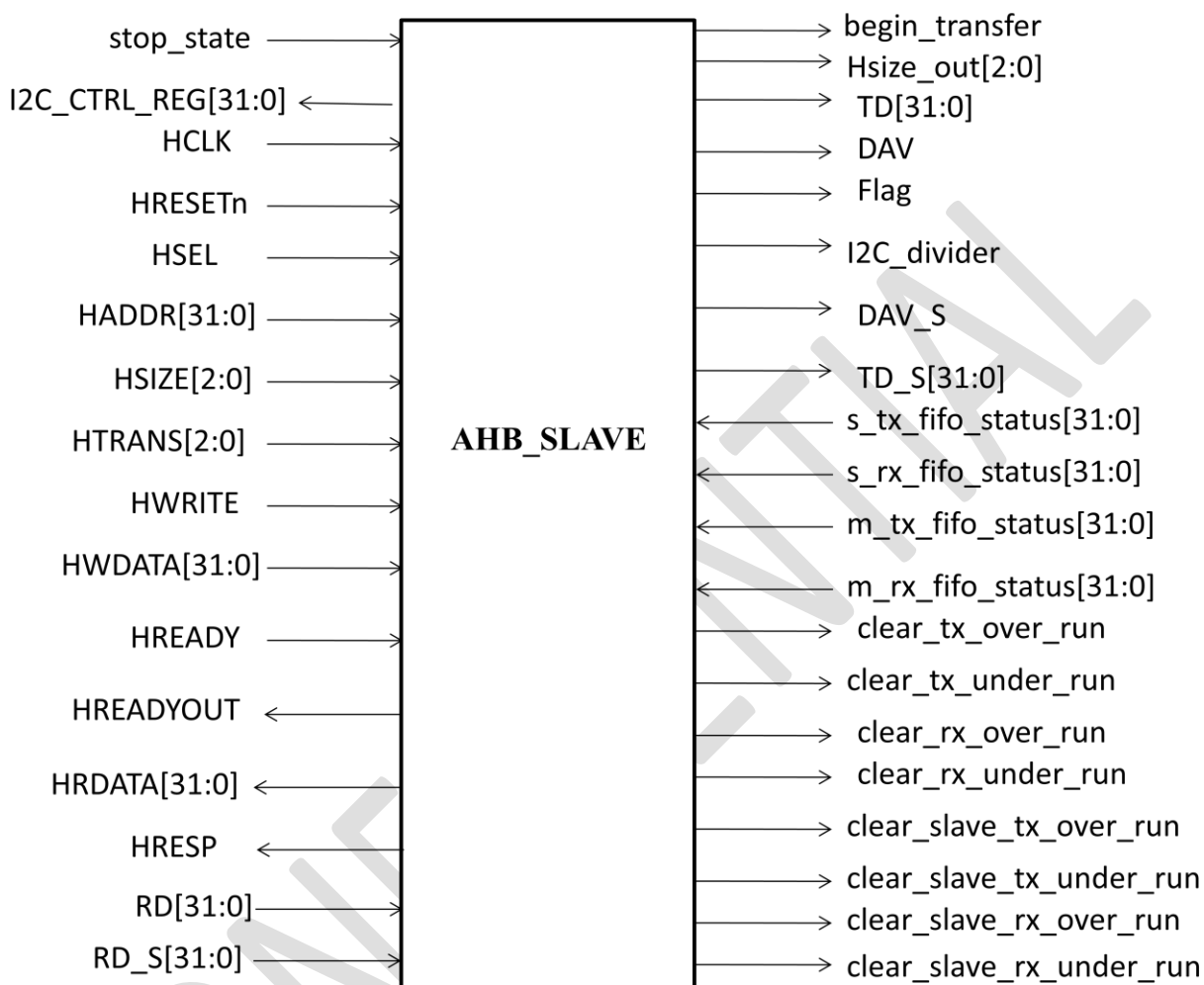


Fig 5: AHB SLAVE

I2C TOP

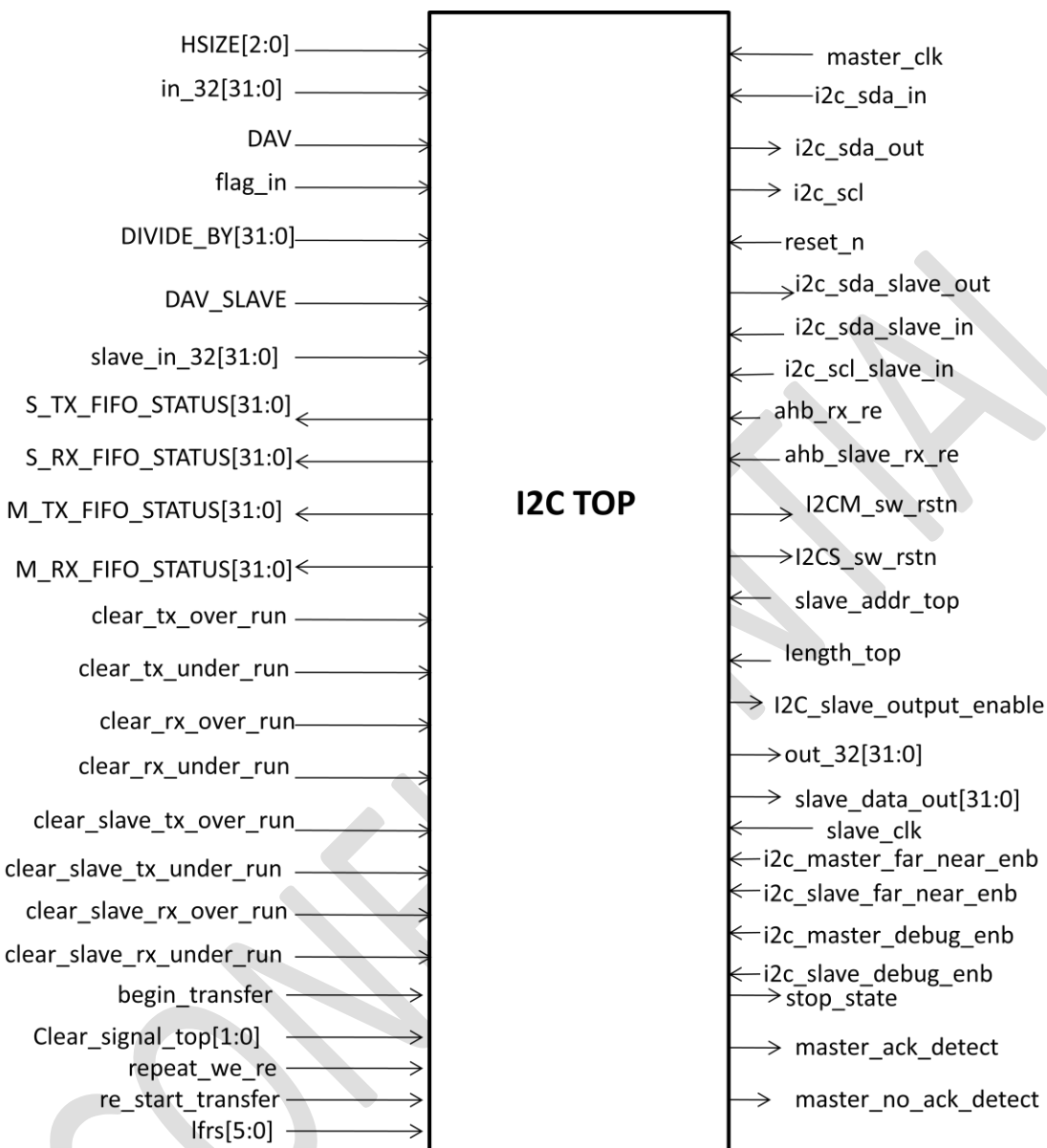


Fig 6: I2C TOP

I2C MASTER

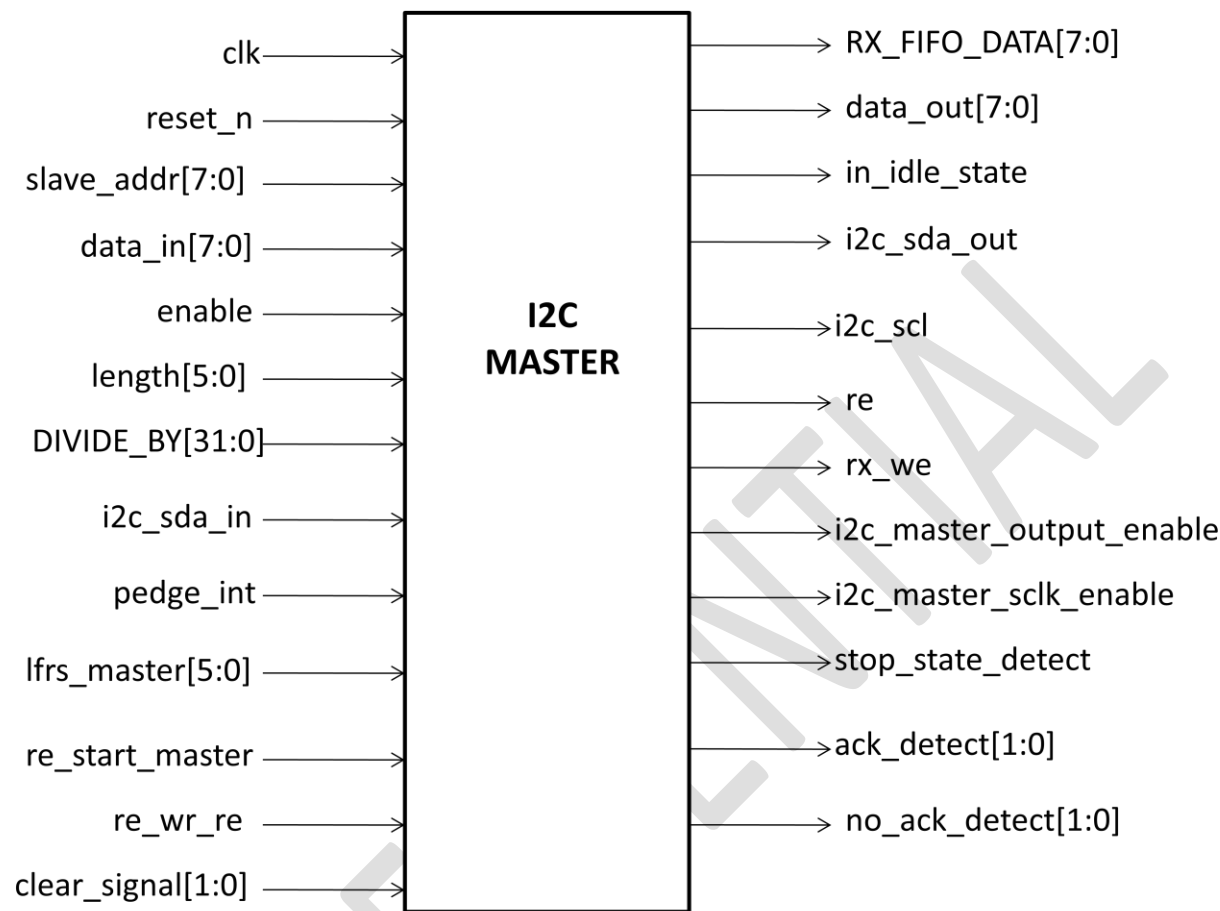


Fig 7: I2C Master

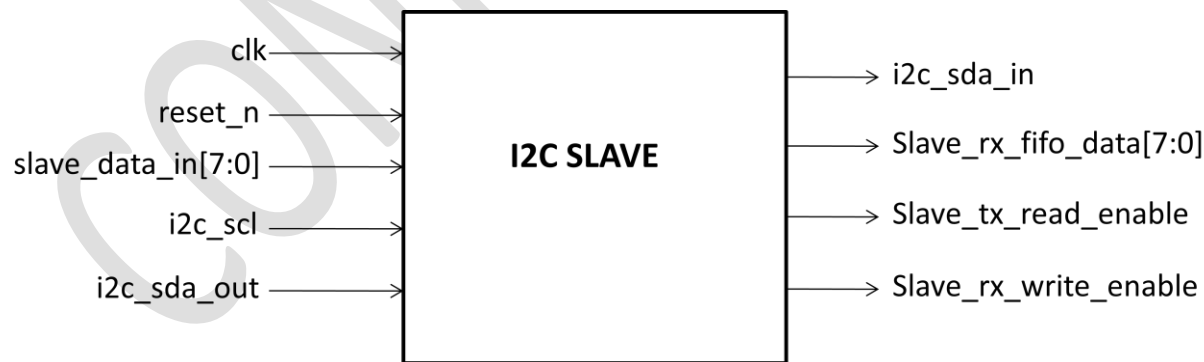


Fig 8: I2C Slave

7. AHB I2C FLOW

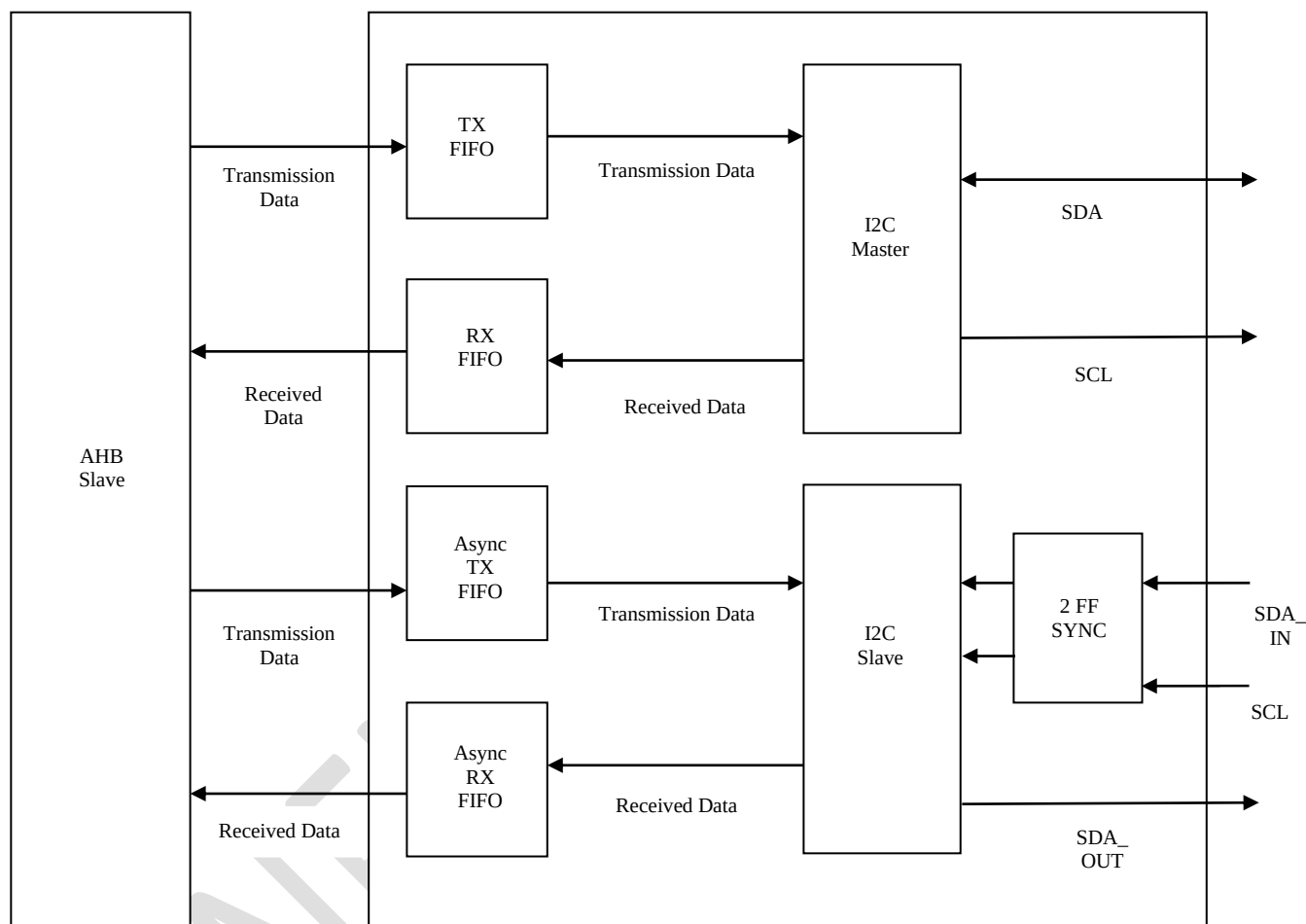


Fig 9: AHB I2C Typical Flow Structure

The Above figure shows the typical flow of AHB I2C structure, the SDA and SCL are the output lines which communicate onto I2C bus through PADS, The following flow requires configuring registers which was described in above chapters i.e., Configuring registers and mode of operation as well as Register Banks.

8. I2C Protocol

A complete transmission on i2c bus includes

- Start and Stop Generation
- Slave Address along with write/read bit
- Acknowledge Bit
- Data Byte

Start and Stop Condition: All transactions begin with a START (S) and are terminated by a STOP (P). A high to low transition on the SDA line while SCL is high defines a start condition. A low to high transition on the SDA line while SCL is high defines a STOP condition. START and STOP conditions are always generated by the master. The bus is considered to be busy after the START condition. The bus is considered to be free again a certain time after the STOP condition.

Address bits followed by read / write bit: After the generation of start master transmits address onto SDA line. I2C supports 7-bit and 10-bit addressing and it is followed by a read/write bit which defines which operation master wants to perform

Acknowledge bit: Once master transmits slave address on the bus. If there is an matching slave with that address that particular slave sends an acknowledge bit during the 9th clock pulse of the SCL

Data byte: Following the acknowledge bit of the selection of slave master transmits data onto SDA line which is 8-bit long only. Numbers of bytes transferred are unrestricted.



Fig 9: I2C Protocol flow

9. Software Reset

The software reset will be coming from AON block and it is an active low signal.

SOFT RESET	i2c_mas_soft_rst_ctrl	0x3A80_0098	8	W/R	0x1
	i2c_slave_soft_rst_ctrl	0x3A80_009C	8	W/R	0x1
	** RSVD for I2C **	0x3A80_00A0	8	W/R	0x1
	spi_mas_soft_rst_ctrl	0x3A80_00A4	8	W/R	0x1
	spi_mas0_soft_rst_ctrl	0x3A80_00A8	8	W/R	0x1
	gpio_soft_rst_ctrl	0x3A80_00AC	8	W/R	0x1
	qspi_soft_rst_ctrl	0x3A80_00B0	8	W/R	0x1
	aesdes_soft_rst_ctrl	0x3A80_00B4	8	W/R	0x1
	arm_wdt_soft_rst_ctrl	0x3A80_00B8	8	W/R	0x1

Since software reset is an active low signal the signal must be set “0” in order to do software rest of I2C – Block for both master and slave.

The software reset signal will have same functionality as HRESETn.

Reference

I2C PROTOCOL DOCUMENT LINK:

<http://web.eecs.umich.edu/~prabal/teaching/resources/eecs373/NXP-I2C-Specification.pdf>

ATEML EEPROM DOCUMENT LINK (Part No of slave used in S7: AT24C01B):

<https://pdf1.alldatasheet.com/datasheet-pdf/view/182249/ATMEL/AT24C01B.html>